Project Proposal: Rocket Telemetry and Image Transmission over TCP

Duygu AKYILDIZ
Student ID: 220303032

Data Communications and Computer Networks Project

22 January 2026

Abstract

This project implements a rocket telemetry and image transmission system using a full-duplex TCP connection. A RocketClient application periodically generates telemetry data such as altitude, velocity, temperature, fuel level and acceleration, and transmits this data to a GroundStationServer. In addition to telemetry information, a JPEG image is transmitted by splitting it into multiple frames. Each frame contains a structured header and a CRC32 checksum for error detection.

A Gateway module is placed between the client and the server to inject controlled random noise into transmitted frames in order to simulate real communication errors. When corrupted frames are detected, the server responds with NACK messages and requests retransmission. This project demonstrates application-level framing, error detection, reliable data transfer and retransmission mechanisms over TCP.

1 Introduction

Telemetry systems are widely used in aerospace and communication applications to monitor critical parameters of vehicles in real time. These systems enable continuous observation of altitude, velocity, temperature and other mission-critical values. In this project, a simulated rocket telemetry system is developed using TCP sockets in the C programming language.

The RocketClient program generates telemetry data periodically and sends it to the GroundStationServer. Additionally, a real image file is transmitted to simulate payload data such as onboard camera images. The use of TCP ensures reliable delivery, while application- level framing is implemented to define message boundaries and metadata.

To make the system more realistic, a Gateway module is introduced between the client and the server. This module injects random bit errors into the transmitted data, allowing the observation of CRC failures and retransmissions. The overall goal of the project is to gain hands-on experience with socket programming, error detection and reliable data transfer.

2 Procedure

The system consists of four main components: RocketClient, Gateway, GroundStationServer, Frame structure and CRC module.

Implementation Steps

1. The client creates a TCP socket and connects to the Gateway using a configurable IP address and port number.
2. Telemetry values are generated periodically using simple mathematical models to simulate changing flight conditions.
3. A JPEG image file is read from disk and divided into multiple frames with a fixed maximum payload size.
4. A CRC32 checksum is calculated for each payload and stored in the frame header.
5. The Gateway forwards frames to the server and optionally injects random bit errors to simulate noise in the communication channel.
6. The server verifies the CRC value of each received frame.
7. If a CRC mismatch is detected, the server sends a NACK message requesting retransmission of the corrupted frame.
8. Correct frames are acknowledged using ACK messages and stored for image reconstruction.

3 Theory

TCP provides reliable, ordered and full-duplex communication between networked applications. It guarantees that data is delivered in the correct order and automatically retransmits lost packets. However, TCP operates as a byte-stream protocol and does not preserve message boundaries.

For this reason, the project implements an application-level frame structure. Each frame contains a header with metadata such as frame type, frame ID, payload size and CRC32 checksum. This approach is similar to higher-level protocols such as HTTP or TLS, which define their own message formats on top of TCP.

CRC32 is a widely used error detection algorithm that detects bit-level corruption in data. By comparing the calculated CRC value with the value stored in the frame header, the server can determine whether a frame has been corrupted during transmission. This allows the system to request retransmission of faulty frames using an ACK/NACK mechanism.

4 Results

The implemented system was tested on a local network environment. Telemetry data was successfully transmitted from the client to the server and logged in real time. Image frames were transmitted, verified and reconstructed correctly on the server side.

When noise injection was enabled in the Gateway module, CRC mismatches occurred and NACK responses were observed. The client successfully retransmitted corrupted frames until they were accepted by the server. These results confirm that the implemented framing and error detection mechanisms function correctly.

Expected outputs achieved in this project include:

- Working TCP client, gateway and server applications implemented in C,
- Logged telemetry data on the server side,
- Successful reconstruction of the transmitted image file,
- Visible CRC errors and retransmissions in program logs.

The diagram in Figure illustrates the structural design of the implemented system using a UML-style representation created with dbdiagram.io. Although dbdiagram.io is primarily designed for database modeling, it was used in this project to represent the main software components and their relationships in a clear and structured manner.

The RocketClient, Gateway and GroundStationServer components are shown together with the Frame and CRCModule structures. The Frame entity represents the core data structure that is exchanged between all components, while the CRCModule is used for error detection. This diagram provides a high-level view of the system architecture and complements the UML class diagram presented earlier.

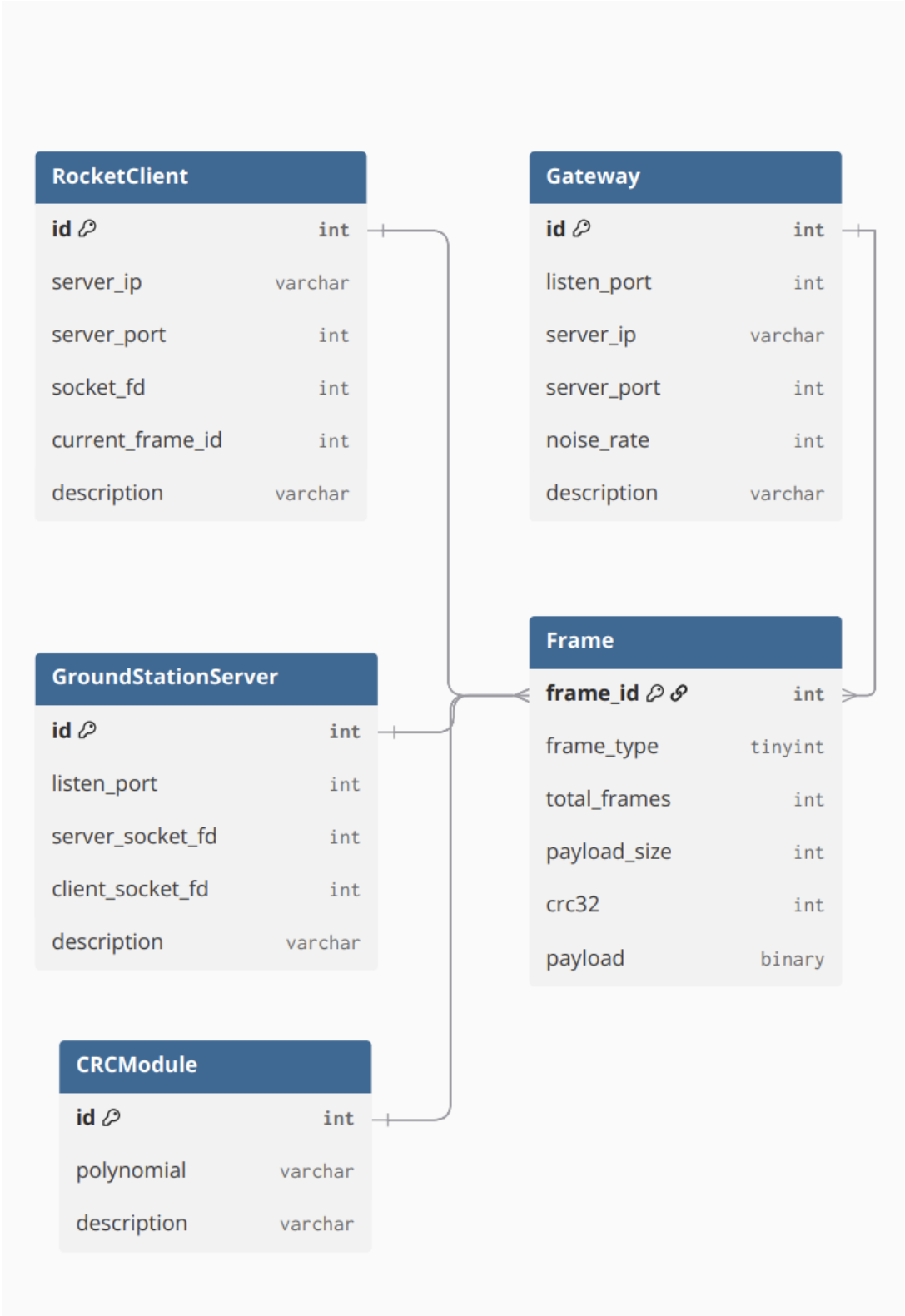

Figure 1: Structural UML-style diagram of the Rocket Telemetry System (dbdiagram.io).

5 Remarks

The project demonstrates the implementation of a reliable telemetry and data transmission system over TCP using application-level framing and error detection. The modular design allows each component to be developed and tested independently.

Possible future improvements include support for multiple clients, encryption of transmitted data, more advanced noise models and graphical visualization of telemetry values and received images in real time.

References

1. A. S. Tanenbaum, *Computer Networks*, Prentice Hall.
2. W. Stallings, *Data and Computer Communications*, Pearson.
3. W. R. Stevens, *UNIX Network Programming*, Addison-Wesley.
4. Beej, "Beej's Guide to Network Programming".
5. J. F. Kurose, K. W. Ross, *Computer Networking: A Top-Down Approach*.
6. RFC 793, Transmission Control Protocol.
7. Istanbul Arel University, BLM431 Lecture Notes.